Guion – Proyecto Integrador: Reserva Salvaje

Integrantes: Juan Suarez

1. Idea del juego

Reserva Salvaje es un juego de gestión en el que el jugador administra una reserva de rescate de animales. Con un presupuesto limitado, rescata animales, los aloja en el hábitat que les corresponde, los alimenta y los cura cuando lo necesitan. Cada tipo de animal tiene una dieta, un hábitat y un costo de mantenimiento distintos, así que el jugador debe equilibrar dinero, espacio disponible y salud de los animales para que la reserva siga funcionando día a día. El objetivo final es recuperar a los animales hasta que estén sanos y poder liberarlos.

2. Clases del modelo y justificación de la jerarquía

Clase base: Animal (abstracta).

Clases hijas: Carnivoro, Herbivoro, Ave, Reptil y AnimalAcuatico.

Qué comparten: todos los animales tienen identidad (id, nombre, edad), un estado (salud y hambre), un hábitat en el que están alojados y la lógica común de ser cuidados (saber si necesitan atención y recibir curación).

Qué las diferencia: cada especie tiene una dieta, un hábitat obligatorio y un costo distintos. Por eso Animal declara métodos abstractos que cada clase hija resuelve a su manera: calcularCostoDiario(), getCostoRescate(), getHabitatRequerido() y alimentar(). Si no existiera la herencia, habría que repetir los atributos comunes en cinco clases o llenar el código de condicionales según el tipo de animal.

Clase	Razón de diseño (qué resuelve que la base no puede)
Carnivoro	Dieta de carne (el costo diario más alto) y obligación de vivir en una jaula de seguridad, separado de otras especies.
Herbivoro	Dieta de forraje, costo diario medio, vive en la sabana junto a otros herbívoros.
Ave	Necesita aviario; su costo depende de la envergadura y de si puede volar.

Clase	Razón de diseño (qué resuelve que la base no puede)
Reptil	Necesita terrario con temperatura controlada; come poco, por lo que su costo diario es bajo.
AnimalAcuatico	Necesita acuario con una profundidad mínima; su costo incluye el mantenimiento del agua.

Clases de apoyo (fuera de la jerarquía): el enum TipoHabitat (SABANA, JAULA_SEGURIDAD, AVIARIO, TERRARIO, ACUARIO) y la clase Reserva, que guarda el presupuesto del jugador.

3. Atributos de cada clase

Clase	Atributos
Animal (base)	id, nombre, edad, salud (0 a 100), hambre (0 a 100), habitat (TipoHabitat en el que el jugador lo aloja)
Carnivoro	los de Animal + kgCarneDiarios
Herbivoro	los de Animal + kgForrajeDiarios
Ave	los de Animal + envergaduraCm, puedeVolar
Reptil	los de Animal + temperaturaIdealC
AnimalAcuatico	los de Animal + profundidadMinimaM
Reserva	id, nombre, presupuesto

4. Diseño de la tabla en MySQL

Se usa una tabla única para toda la jerarquía, con la columna tipo como discriminador. Las columnas propias de cada subclase quedan en NULL para los animales a los que no aplican. El DAO lee la columna tipo para decidir qué clase instanciar. Además hay una tabla pequeña con una única fila para el presupuesto de la reserva.

```
CREATE DATABASE IF NOT EXISTS reserva_salvaje;
USE reserva_salvaje;

CREATE TABLE animal (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tipo VARCHAR(20) NOT NULL,
```

```
    nombre VARCHAR(50) NOT NULL,
    edad INT NOT NULL,
    salud INT NOT NULL,
    hambre INT NOT NULL,
    habitat VARCHAR(20) NOT NULL,
    kg_carne DECIMAL(5,2) NULL,
    kg_forraje DECIMAL(5,2) NULL,
    envergadura_cm INT NULL,
    puede_volar BOOLEAN NULL,
    temperatura_ideal DECIMAL(4,1) NULL,
    profundidad_minima DECIMAL(5,2) NULL,
    CONSTRAINT chk_tipo CHECK (tipo IN
('CARNIVORO','HERBIVORO','AVE','REPTIL','ACUATICO')),
    CONSTRAINT chk_edad CHECK (edad >= 0),
    CONSTRAINT chk_salud CHECK (salud BETWEEN 0 AND 100),
    CONSTRAINT chk_hambre CHECK (hambre BETWEEN 0 AND 100)
);

CREATE TABLE reserva (
    id INT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    presupuesto DECIMAL(10,2) NOT NULL,
    CONSTRAINT chk_presupuesto CHECK (presupuesto >= 0)
);

INSERT INTO reserva (id, nombre, presupuesto) VALUES (1, 'Reserva Salvaje',
1000.00);
```

5. Reglas de negocio y excepciones

Nº	Regla que valida el service	Excepción que dispara
1	Rescatar, alimentar o curar un animal requiere presupuesto suficiente.	FondosInsuficientesException
2	Cada hábitat tiene una capacidad máxima (sabana 4, jaula de seguridad 2, aviario 3, terrario 3, acuario 2). No se puede rescatar un animal si su hábitat está lleno.	HabitatLlenoException
3	El animal debe alojarse en el hábitat que exige su especie (getHabitatRequerido). Por eso un carnívoro nunca comparte hábitat con otras especies y un ave no puede ir al acuario.	HabitatIncompatibleException

Nº	Regla que valida el service	Excepción que dispara
4	Toda operación sobre un id que no existe en la base se rechaza.	AnimalNoEncontradoException
5	Solo se puede liberar a un animal si su salud es de 60 o más.	AnimalEnfermoException
6	Al rescatar se validan los datos: nombre no vacío, edad no negativa, salud y hambre entre 0 y 100, hábitat no nulo.	DatosInvalidosException
7	Cualquier SQLException se captura en el DAO y se relanza envuelta, para que el Main nunca reciba un error sin controlar.	PersistenciaException

Además, al avanzar el día aumenta el hambre de todos los animales, los que superan un umbral de hambre pierden salud, y la reserva recibe donaciones según la cantidad de animales sanos. Así el jugador tiene ingresos y presión para cuidar a los animales.

6. Interfaz de comportamiento común y uso polimórfico

Interfaz: Cuidable. Define lo que cualquier animal debe poder hacer para ser atendido:

- alimentar()
- curar(int puntos)
- necesitaAtencion()
- calcularCostoDiario()

Animal la implementa y cada subclase concreta alimentar() según su dieta (por ejemplo, cuánto hambre calma una ración de carne frente a una de forraje o de alimento acuático).

Uso polimórfico en el service: el método atenderAnimal(Cuidable cuidable) recibe el tipo interfaz y opera sin importar si le llega un Carnivoro, un Ave o cualquier otra subclase: si necesitaAtencion() es verdadero, verifica fondos con calcularCostoDiario(), llama a alimentar() y, si hace falta, a curar(). Luego el service persiste el cambio mediante el DAO. El método atenderTodos() recorre los animales de la reserva y llama a atenderAnimal() con cada uno.

7. Métodos previstos del service (ReservaService)

- rescatarAnimal(Animal animal)
- buscarAnimal(int id)
- listarAnimales()

- listarPorHabitat(TipoHabitat habitat)
- alimentarAnimal(int id)
- curarAnimal(int id)
- atenderAnimal(Cuidable cuidable) — método polimórfico
- atenderTodos()
- liberarAnimal(int id)
- avanzarDia()
- consultarPresupuesto()

Anexo: estructura prevista del proyecto

- model: Animal, Carnivoro, Herbivoro, Ave, Reptil, AnimalAcuatico, Reserva, TipoHabitat, Cuidable
- dao: ConexionDB, AnimalDAO, ReservaDAO
- service: ReservaService
- exception: las siete excepciones de la sección 5
- main: Main, que recorre el flujo completo (rescatar distintos tipos de animales, alimentar, curar, atender a todos, avanzar el día y liberar) y fuerza a propósito cada una de las excepciones para mostrar que se controlan.